



# CONSTRAINTS AND TRIGGERS

SHENKAR

## CONSTRAINTS

ישנם סוגים שונים של אילוצים.  
ניתן לחלקם ל 2 קבוצות:

- Table Constraints
- Referential Integrity Constraints

## TABLE CONSTRAINTS

- By default, SQL tables have ***no*** constraints
  - You can insert multiple copies of a given row
  - You can insert rows with **NULL** values

## PRIMARY KEY CONSTRAINTS

– לא ניתן להכניס שתי רשומות עם אותן ערכים ל PK  
– טבלה יכול להיות PK אחד בלבד

```
CREATE TABLE account (  
    account_number CHAR(10) ,  
    branch_name VARCHAR(20) ,  
    Balance NUMERIC(12, 2) ,  
    PRIMARY KEY (account_number)  
);
```

Or

```
CREATE TABLE account (  
    account_number CHAR(10) PRIMARY KEY ,  
    branch_name VARCHAR(20) ,  
    Balance NUMERIC(12, 2)  
);
```

## PRIMARY KEY CONSTRAINTS

- For multi-column primary keys, must specify primary key after column specifications

```
CREATE TABLE depositor (  
    customer_name VARCHAR(30),  
    account_number CHAR(10),  
    PRIMARY KEY (customer_name, account_number)  
);
```

## NULL-VALUE CONSTRAINTS

- ניתן להכניס NULL לעמודה – זוהי ברירת המחדל (default).
- לא תמיד נרצה שזה יקרה.  
– לדוגמה:  
בחשבון בנק תמיד נרצה שיהיה ערך ביתרה.
- ניתן להגדיר אילוץ **NOT NULL** על מנת לא לאפשר NULL בעמודה מסויימת

```
CREATE TABLE account (  
    account_number CHAR(10) PRIMARY KEY ,  
    branch_name VARCHAR(20) NOT NULL,  
    Balance NUMERIC(12, 2) NOT NULL  
);
```

## UNIQUE CONSTRAINTS

- ישנן טבלאות שיש להן יותר ממפתח אחד פוטנציאלי
- את המפתחות הנוספים נגדיר באמצעות האילוץ

### UNIQUE

- Like primary key constraints, can specify in the column declaration, or after all columns
- Can only specify multi-column candidate key after the column specifications

- בניגוד ל PK ה UNIQUE אינו מונע הכנסת NULL

- This constraint considers **NULL** values to be unequal!

## UNIQUE CONSTRAINTS - EXAMPLE

```
CREATE TABLE employee (  
  emp_id INT PRIMARY KEY,  
  emp_ssn CHAR(9) NOT NULL UNIQUE,  
  emp_name VARCHAR(40) NOT NULL,  
  ...  
);
```

- Employee's ID is the primary key
- All employees need SSN (social number), but no two employees should have the same SSN
  - Don't forget **NOT NULL** constraint too!
- All employees should have a name, but multiple employees might have same name



## UNIQUE AND NULL - EXAMPLE

```
CREATE TABLE customer (  
    cust_name  VARCHAR(30) NOT NULL,  
    address    VARCHAR(60) ,  
    UNIQUE (cust_name, address)  
);
```

- Try inserting values:

```
INSERT INTO customer  
VALUES ('John Doe', '123 Spring Lane');
```

```
INSERT INTO customer  
VALUES ('John Doe', '123 Spring Lane');
```

- Second insert fails, as expected

## CHECK CONSTRAINTS

- Often want to specify other constraints on values
- Can require values in a table to satisfy some predicate, using a **CHECK** constraint
  - Very effective for constraining columns' domains, and eliminating obviously bad inputs
- **CHECK** constraints must appear after column specifications

## CHECK CONSTRAINT EXAMPLES

- ניתן להוסיף אילוץ על עמודה מסויימת:

```
CREATE TABLE employee (  
    emp_id    INT           PRIMARY KEY,  
    emp_ssn   CHAR(9)       NOT NULL UNIQUE,  
    emp_name  VARCHAR(40)   NOT NULL,  
    pay_rate  NUMERIC(5,2)  NOT NULL,  
    CHECK (pay_rate > 5.25)  
);
```

- האילוץ יבדוק שהשכר מעל ערך מסוים

## EXAMPLE

```
CREATE TABLE employee (  
    emp_id    INT          PRIMARY KEY,  
    emp_ssn   CHAR(9)      NOT NULL UNIQUE,  
    emp_name  VARCHAR(40)  NOT NULL,  
    status    VARCHAR(10)  NOT NULL,  
    pay_rate  NUMERIC(5,2) NOT NULL,  
    CHECK (pay_rate > 5.25),  
    CHECK (status IN  
           ('active', 'vacation', 'suspended'))  
);
```

- סטטוס העובד חייב להיות אחד מהשלושה המוגדרים באילוץ

## ANOTHER CHECK CONSTRAINT

```
CREATE TABLE depositor (  
    customer_name VARCHAR(30) ,  
    account_number CHAR(10) ,  
    PRIMARY KEY (customer_name, account_number) ,  
    CHECK (account_number IN  
           (SELECT account_number FROM account))  
);
```

- Rows in depositor table should only contain valid account numbers!
  - The valid account numbers appear in account table
  - This is a referential integrity constraint

# REFERENTIAL INTEGRITY CONSTRAINTS

- Most important constraints are referential integrity constraints
  - These constraints span multiple tables
  - One table's values are constrained by another table's values
- A relation can include attributes of another relation's primary key
  - Called a foreign key

# FOREIGN KEY EXAMPLE

```
CREATE TABLE employee (  
    emp_id    INT           PRIMARY KEY,  
    emp_ssn   CHAR(9)       NOT NULL UNIQUE,  
    emp_name  VARCHAR(40)   NOT NULL,  
    ...  
);
```

- Health plan information:

```
CREATE TABLE health_plan (  
    emp_ssn CHAR(9) PRIMARY KEY,  
    provider VARCHAR(20) NOT NULL,  
    FOREIGN KEY (emp_ssn) REFERENCES  
    employee (emp_ssn));
```

## MULTIPLE CONSTRAINTS

- Can combine several different constraints

```
emp_ssn CHAR(9) PRIMARY KEY  
                FOREIGN KEY (emp_ssn)  
                REFERENCES employee (emp_ssn))
```

- *emp\_ssn* is primary key of *health\_plan* relation
- *emp\_ssn* is also a foreign key to *employee* relation
- Foreign key references the candidate-key *employee.emp\_ssn*



## SELF-REFERENCING FOREIGN KEYS

- מפתח זר יכול להצביע לעצמו (לא רק לטבלה אחרת)  
בדרך כלל כשנרצה לממש מבנה היירכי

```
CREATE TABLE employee (  
  emp_id INT      PRIMARY KEY,  
  emp_name VARCHAR(40) NOT NULL,  
  manager_id int,  
  FOREIGN KEY (manager_id) REFERENCES  
  employee(emp_id) );
```

- Allow manager ID to be **NULL** for employees with no manager

## FOREIGN KEY VIOLATIONS

- ישנן מספר דרכים להפר את אילוצי המפתח הזר.
- הכנסת/ עדכון ערך לא נכון:
  - e.g. trying to insert a row into *health\_care* relation, with an invalid emp\_ssn
  - e.g. trying to update a row in *health\_care* relation, to have an invalid emp\_ssn
- מה יקרה אם ננסה למחוק רשומה שיש אליה מפתח זר
  - What to do with *health\_care* if a row is deleted from *employee*?

## EXAMPLE

- **employee :**

```
CREATE TABLE employee (  
  emp_id INT PRIMARY KEY,  
  emp_ssn CHAR(9) NOT NULL UNIQUE,  
  emp_name VARCHAR(40) NOT NULL,  
  pay_rate NUMERIC(5 , 2 ) NOT NULL,  
  CHECK (pay_rate > 5.25));  
);
```

- **health\_plan :**

```
CREATE TABLE health_plan (  
  emp_ssn CHAR(9) PRIMARY KEY,  
  provider VARCHAR(20) NOT NULL,  
  FOREIGN KEY (emp_ssn) REFERENCES  
  employee (emp_ssn));
```

- *employee*  
data:

|   | emp_id | emp_ssn | emp_name  | pay_rate |
|---|--------|---------|-----------|----------|
| ▶ | 1      | 666     | Jane Doe  | 6.00     |
|   | 2      | 777     | Jhone Doe | 5.50     |

- *health\_plan*  
data:

|   | emp_ssn | provider |
|---|---------|----------|
| ▶ | 666     | AI       |
|   | 777     | AI       |

Want to delete `emp_id = 1` from *employee*.  
What should happen?

## FOREIGN KEY VIOLATIONS

- Option 1: Disallow the delete from *employee*
  - Default for SQL.
- Option 2: Cascade the delete operation
  - If user deletes **emp\_id = 1** from *employee* relation, *all* referencing rows in *health\_care* should also be deleted

# CASCADING CHANGES

- Specify behavior in these situations on the foreign key constraint

```
CREATE TABLE health_plan (  
  emp_ssn CHAR(9),  
  provider VARCHAR(20) NOT NULL,  
  FOREIGN KEY (emp_ssn)  
  REFERENCES employee (emp_ssn)  
  ON DELETE CASCADE ON UPDATE CASCADE);
```

- When `emp_id = 1` is deleted from *employee* relation, corresponding rows in *health\_care* will be deleted too
- Similar considerations for updates to primary key values in referenced relation
  - Can also specify **ON UPDATE** behaviors

# TRIGGERS

- Specify automatic actions that database system will perform when certain events and conditions occur
- Triggering event can be **insert**, **delete** or **update**
- Values of attributes before and after an *update* can be referenced
  - ***referencing old row as*** : for deletes and updates
  - ***referencing new row as*** : for inserts and updates

## EXAMPLE

```
CREATE TRIGGER ins_employee AFTER
INSERT ON employee FOR EACH ROW
BEGIN
INSERT INTO health_plan
(emp_ssn,provider)
VALUES (new.emp_ssn, 'AI') ;
END;
```



## EXAMPLE

```
CREATE TRIGGER upd_employee AFTER
UPDATE ON employee FOR EACH ROW
BEGIN
IF (old.pay_rate != new.pay_rate)
THEN
INSERT INTO pay_rate_log (emp_id,
old_pay_rate, new_pay_rate)
VALUES
(old.emp_id, old.pay_rate, new.pay_rate
);
END IF;
END
```

# תרגיל

- צרי/צור בסיס נתונים חדש – shenkar
- בתוכו:

- student (s\_id: INT, s\_name: STRING, s\_address: STRING, s\_status: STRING)  
s\_id – PK  
s\_status – with default value
- lecturer (l\_id: INT, l\_name: STRING, dept\_id: DEPTS)  
l\_id - PK
- course (dept\_id: DEPTS, c\_name: STRING, c\_code: INT, c\_descr: STRING)  
c\_code – PK  
c\_name,dept\_id - UNIQUE
- register (c\_code: COURSE, s\_id: STUDENT, grade : int, semester: int)  
c\_code,s\_id,semester - PK
- department(d\_code: INT, d\_name: STRING, d\_descr: STRING )  
d\_code – PK, d\_name - UNIQUE

- יש להכניס לכל טבלה 5 רשומות (ההכנסה צריכה להיות בסדר הנכון) – יש להכניס נתונים כך שיתקבלו תוצאות לכל השאילתות הבאות.

- שאילתות:

- שם הסטודנט, שם הקורס, מספר סמסטר והציון בו.
- שם הקורס בו יש הכי הרבה נרשמים ומה מספרם.
- שם הקורס אליו לא נרשמו.
- רשימת המרצים והקורסים המשויכים למחלקה שלהם (p\_name, c\_name)

- עדכון נתונים: יש להוסיף 10 נקודות לכל הסטודנטים.

- מחיקת נתונים: מחק/י את קורס מספר 1 (המחיקה צריכה להיות בסדר הנכון)

- הוספי/הוסף

1. טבלת register\_log הטבלה תהיה במבנה של טבלת register ובנוסף עמודת  
change\_time datetime) change\_action char(1)
2. טריגר על טבלת register שירשום כל שינוי בנתוני הטבלה לטבלת register\_log